UCG
Univerzitet Crne Gore

OPERATIVNI SISTEMI

Docker: šta je kontejner

Motivacija, slika i kontejner, namespace i cgroup

Velibor Došljak

Plan za četiri časa

1. **Šta je kontejner** (danas): motivacija, slika i kontejner, namespace i cgroup, prve komande
2. **Slike i Dockerfile**: `build`, `RUN` i `CMD`, kontekst, slojevi
3. **Podaci i pokretanje**: volume, bind mount, korisnici, portovi, resursi, signali
4. **Docker Compose**: više kontejnera, realna web aplikacija

Šta nam treba

C, osnove shell-a i Linux VM sa kursa. Python, baze i web ćemo sretati u primjerima, ali ih ne moramo znati: bitno je kako se takva aplikacija pakuje i pokreće.

01

Motivacija



Motivacija

- „Kod mene radi“ je jedan od najčešćih problema u razvoju softvera
- Program zavisi od svog okruženja:
  - biblioteka i njihovih verzija
  - alata i konfiguracije
  - operativnog sistema
- Ideja: zajedno sa programom isporučimo i okruženje



Pokretanje tuđe aplikacije

Bez Dockera

- Instaliraj ove alate
- Instaliraj DBMS (sistem za upravljanje bazom podataka)
- Instaliraj ove biblioteke
- Promijeni ovaj konfiguracioni fajl
- I dalje ne radi: koristim Windows, a ovo je pravljeno za Linux

Sa Dockerom

```
$ docker compose up
```

Da budemo precizni

Linux kontejneru treba Linux kernel. Na Windows-u i macOS-u Docker Desktop zato pokreće Linux u virtuelnoj mašini.

Demo 1: isti problem u C-u

Običan program, kompajliran na novijem sistemu, pa pokrenut na drugim sistemima:

```c
#include <stdio.h>
int main(){ printf("Radi kod mene!\n"); return 0; }
```

```
$ ./program                # stariji Ubuntu 20.04
./program: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_2.34' not found
$ ./program                # Alpine, koji nema glibc
exec ./program: no such file or directory
```

- Fajl `program` postoji i u drugom slučaju. Ne postoji njegov dinamički linker!

Zašto? Dinamičko povezivanje

- Izvršni fajl sadrži samo naš kod
- `printf` je u biblioteci `libc.so.6` koja mora postojati na sistemu, u odgovarajućoj verziji
- `gcc -static` ugrađuje biblioteke u fajl: 15 KB postaje 935 KB, ali radi svuda
- Realne aplikacije zavise od mnogo više: interpretera, baza, alata

Ideja kontejnera

Spakujemo program **i sve od čega zavisi**, osim kernela. Kernel je zajednički, a njegov interfejs (sistemski pozivi) je stabilan.

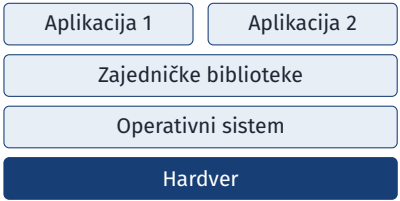
02

Evolucija virtuelizacije

Bare metal

Aplikacije direktno na operativnom sistemu

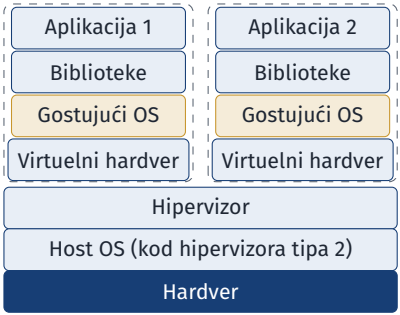
- Konflikti između verzija softvera
- Slaba iskorišćenost hardvera
- Slaba izolacija: problem u jednoj aplikaciji utiče na druge
- Sporo restartovanje sistema
- Zahtjevna instalacija na novi hardver



Virtuelne mašine

Svaka aplikacija sa svojim operativnim sistemom

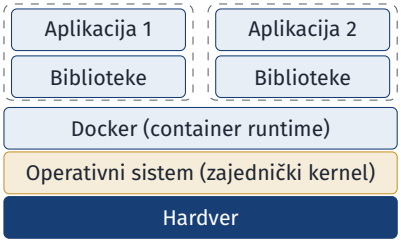
- Hipervizor dijeli resurse hosta i simulira više odvojenih mašina
- Nema konflikta između verzija softvera
- Bolja iskorišćenost hardvera
- Jaka izolacija: svaka VM ima svoj kernel
- Cijena: svaka VM nosi cijeli OS (GB memorije i diska)



Kontejneri

Aplikacije dijele kernel hosta

- Kontejner ne pokreće svoj kernel, već koristi kernel host sistema
- Izolaciju obezbjeđuju mogućnosti Linux-a: namespace, cgroup, union fajl sistem
- Nema konflikta između verzija softvera
- Još bolja iskorišćenost hardvera
- Pokretanje je gotovo trenutno



Virtuelna mašina ili kontejner?

	Virtuelna mašina	Kontejner
Kernel	svoj	zajednički sa hostom
Izolacija	jača	slabija
Pokretanje	sekunde do minuti	obično djelić sekunde
Veličina	obično GB	od nekoliko MB
Drugi OS	i Windows na Linux-u	traži Linux kernel

Kontejner nije „bolja VM“

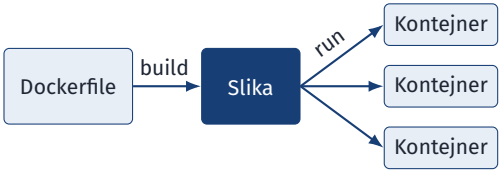
Ako je problem u kernelu, pogođeni su svi kontejneri. U praksi se koriste oba: kontejneri često rade unutar virtuelnih mašina.

03

Slika i kontejner

Slika i kontejner

- **Slika** (*image*): paket sa svim što je potrebno aplikaciji; ne mijenja se
- **Kontejner**: pokrenut, izolovan proces nastao iz slike
- Iz jedne slike možemo napraviti više kontejnera



Poređenje sa C-om

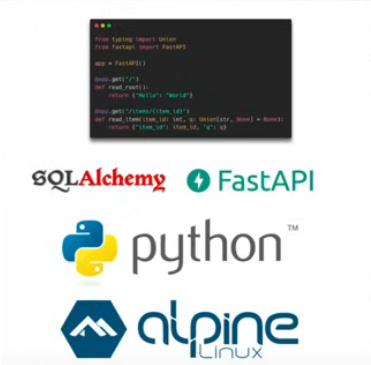
Slika je kao izvršni fajl na disku, a kontejner kao proces koji iz njega nastaje.

Šta se nalazi u slici?

- OS paketi (ali ne i kernel)
- Runtime okruženje (npr. Python ili Node.js)
- Sve potrebne biblioteke
- Kod aplikacije
- Osim Dockera nije potreban drugi softver
- Lako pokrećemo više instanci aplikacije

Naš prvi primjer, u C-u:
`ubuntu:22.04` + `gcc` + `main.c` + izvršni fajl

Realan primjer: web API u Pythonu



04

Kontejner je proces

Šta kontejner čini kontejnerom?

Kontejner nije posebna vrsta mašine, već običan Linux proces kojem je kernel ograničio **šta vidi** i **koliko smije da koristi**.

Mehanizam	Čemu služi	Primjer
namespace	šta proces *vidi*	svoje PID-ove, hostname, mrežu, mount-ove
cgroup	koliko *smije da koristi*	najviše 50 MB memorije, 20 procesa
union fajl sistem (OverlayFS)	kako se slažu slojevi slike	slika se ne mijenja, a kontejner dodaje svoj sloj

Demo 2: kako proces vidi sistem

```c
#include <sys/utsname.h>     // i stdio.h, stdlib.h, unistd.h

int main(int argc, char *argv[]){
    struct utsname u;
    uname(&u);
    printf("PID=%d PPID=%d UID=%d\n", getpid(), getppid(), getuid());
    printf("hostname=%s\n", u.nodename);
    printf("kernel=%s\n", u.release);
    fflush(stdout);
    if(argc > 1)
        sleep(atoi(argv[1]));
    return 0;
}
```

Demo 2: host i kontejner

```
$ ./proces                                                    # na hostu
PID=256783 PPID=256126 UID=1000
hostname=asus-tuf-a14
kernel=7.3.0-10-generic
$ docker run --rm -v "$PWD":/w -w /w ubuntu:22.04 ./proces    # u kontejneru
PID=1 PPID=0 UID=0
hostname=2ab4afb97326
kernel=7.3.0-10-generic
```

- U kontejneru je naš program **PID 1**, ima drugi hostname i radi kao root
- Kernel je isti, jer je kernel zajednički
- Opciju `-v` detaljno obrađujemo na trećem času: ovdje samo unosi fajl sa hosta u kontejner

Demo 2: isti proces viđen sa hosta

```
$ docker run -d --name demo-proces -v "$PWD":/w -w /w ubuntu:22.04 ./proces 40
$ docker logs demo-proces | head -1
PID=1 PPID=0 UID=0
$ docker top demo-proces -o pid,ppid,cmd
PID       PPID      CMD
256921    256897    ./proces 40
$ docker inspect -f '{{.State.Pid}}' demo-proces
256921
```

- Isti proces ima **dva PID-a**: unutra 1, na hostu 256921
- Vidi se i običnom komandom `ps` na hostu: nema „mašine“ oko njega

Demo 3: mini kontejner u C-u

Ono što Docker radi ispod haube, bez Dockera (skraćeno):

```c
static char stek[1024 * 1024];
static int kontejner(void *arg){
    char **argv = arg;
    sethostname("kontejner", 9);
    mount(NULL, "/", NULL, MS_REC | MS_PRIVATE, NULL);   // mount-ovi ne cure na host
    mount("proc", "/proc", "proc", 0, NULL);             // ps vidi samo ovaj namespace
    printf("[KONTEJNER] PID=%d\n", getpid());
    execvp(argv[1], &argv[1]);
}
int main(int argc, char *argv[]){
    int zastavice = CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | SIGCHLD;
    pid_t pid = clone(kontejner, stek + sizeof(stek), zastavice, argv);
    waitpid(pid, NULL, 0);
}
```

Demo 3: rezultat

```
$ sudo ./mini_kontejner ps -ef
[HOST] PID=1 hostname=host-masina
[HOST] dijete je sa moje strane PID=7
[KONTEJNER] PID=1 hostname=kontejner
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 16:07 ?        00:00:00 ps -ef
[HOST] kontejner je zavrsio, hostname je i dalje host-masina
```

- `ps` vidi samo sebe, a hostname je promijenjen samo za dijete
- Sa strane roditelja, dijete ima drugi PID
- Docker dodaje još: slike, slojeve, mrežu, cgroup ograničenja, …

Izlaz je sniman unutar privilegovanog kontejnera, pa su PID-ovi hosta mali. Pokrenut sa `sudo` na hostu, program prikazuje stvarne PID-ove hosta.

05

Prve komande

Instalacija Docker Engine-a

- Linux virtuelna mašina koju ste koristili u prvom dijelu kursa
- Uputstvo za instalaciju za vaš OS:
  `https://docs.docker.com/engine/install/`

Provjera instalacije:

```
$ docker run hello-world
```

Ako se javi `permission denied` na `docker.sock`, korisnika dodajemo u grupu `docker` i ponovo se prijavljujemo:

```
$ sudo usermod -aG docker $USER
```

Pažnja: članstvo u grupi `docker` praktično znači root pristup hostu. Radite ovo samo na svojoj mašini.

Pokretanje kontejnera

```
$ docker run --interactive --tty --rm ubuntu:22.04
```

`docker run`	napravi kontejner iz slike i pokreni ga
`--interactive`, `-i`	drži otvoren standardni ulaz (`stdin`)
`--tty`, `-t`	dodijeli terminal; zajedno daju interaktivni shell
`--rm`	**ukloni** kontejner čim se završi
`ubuntu:22.04`	slika od koje se pravi kontejner (ime i tag)

```
$ docker ps          # pokrenuti kontejneri
$ docker ps --all    # i zaustavljeni
```

Unutar kontejnera

```
root@2ab4afb97326:/# ping google.com -c 1
bash: ping: command not found
root@2ab4afb97326:/# apt update
root@2ab4afb97326:/# apt install -y iputils-ping
root@2ab4afb97326:/# ping google.com -c 1
root@2ab4afb97326:/# exit
```

- Slika sadrži samo minimum, pa `ping` nije instaliran
- `apt` je alat za instalaciju softvera na Debian i Ubuntu sistemima
- `exit` završava `bash`. Pošto je on **PID 1**, završava se i kontejner

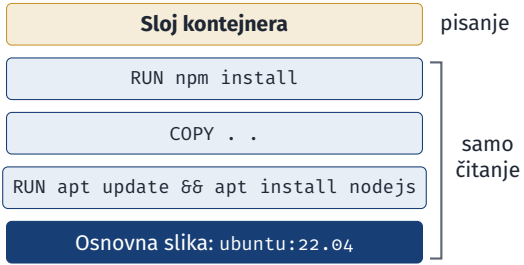
Probajte ponovo

U novom `docker run` kontejneru `ping` opet ne postoji. Zašto?

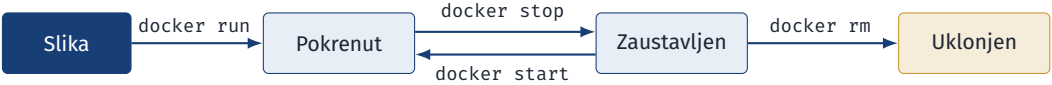
Fajl sistem kontejnera

- Slika se sastoji od slojeva koji se **samo čitaju**
- Svaki kontejner dobija svoj prazan sloj za pisanje
- Slika ostaje ista, pa dijeljenje slojeva štedi prostor
- Novi kontejner znači novi sloj: zato `ping` nestaje

Primjer desno: slika za aplikaciju u Node.js-u



Životni ciklus kontejnera



Podaci se brišu pri uklanjanju, ne pri zaustavljanju

- Sloj za pisanje postoji sve dok kontejner nije uklonjen
- Sa `docker stop` i `docker start` fajlovi ostaju
- Sa `docker rm` nestaju; opcija `--rm` to radi automatski čim se kontejner završi

Kontejner koji se čuva nakon zaustavljanja

```
$ docker run -it --name my-ubuntu-container ubuntu:22.04
```

- `-it` je isto što i `--interactive --tty`
- Nema `--rm`. Bez `--name` Docker dodjeljuje nasumično ime

U drugom terminalu, jer je prvi zauzet shell-om iz kontejnera:

```
$ docker stop my-ubuntu-container
$ docker ps -a | grep my-ubuntu-container
$ docker inspect my-ubuntu-container
$ docker start my-ubuntu-container
$ docker attach my-ubuntu-container          # povezi se na PID 1 (bash)
$ docker exec -it my-ubuntu-container bash   # novi proces u kontejneru
```

Iz `attach` izlazimo sa `Ctrl-P Ctrl-Q`, jer `exit` zaustavlja kontejner. `exit` iz `exec` završava samo taj novi proces.

Demo 4: sloj kontejnera

```
$ docker run -d --name demo-sloj ubuntu:22.04 sleep 300
$ docker exec demo-sloj sh -c 'echo x > /f.txt'
$ docker diff demo-sloj                  # sta je promijenjeno u odnosu na sliku
A /f.txt
$ docker stop demo-sloj
$ docker start demo-sloj
$ docker exec demo-sloj cat /f.txt       # fajl je prezivio stop i start
x
$ docker rm -f demo-sloj
$ docker run --rm ubuntu:22.04 cat /f.txt
cat: /f.txt: No such file or directory
```

- Slika se nije promijenila, sve je bilo u sloju tog kontejnera
- `docker rm -f` zaustavlja i uklanja kontejner

Rezime

- Slika je paket, a kontejner pokrenut proces
- Kontejner dijeli kernel sa hostom
- namespace određuje šta proces vidi, a cgroup koliko smije da koristi
- Podaci se brišu uklanjanjem kontejnera

Komande

`docker run`, `ps -a`, `stop`, `start`, `attach`, `exec`, `logs`, `top`, `diff`, `inspect`, `rm`

Za vježbu

Pokrenite `proces` u dva kontejnera. Koliki je PID unutra, a koliki na hostu?

Izvori

- Izbor tema i dio primjera: DevOps Directive Docker Course
  `https://github.com/sidpalas/devops-directive-docker-course`
- Dokumentacija: `https://docs.docker.com`
- Dockerfile referenca: `https://docs.docker.com/reference/dockerfile/`
- Open Container Initiative: `https://opencontainers.org`

Hvala na pažnji

Sljedeći čas: slike i Dockerfile

UCG
Univerzitet Crne Gore